WebAssemblyクリプトジャッキングマルウェアにおける難読化技術の評価

貝沼英亮[1,a)]　向井宏明[1,b)]

概要：WebAssembly（Wasm）は，Webブラウザ上で高速な処理を実現する技術である．一方で，その処理性能を悪用し，利用者の同意なく端末の計算資源を用いて暗号資産をマイニングする「クリプトジャッキング」が問題となっている．また，コードの構造を複雑化する難読化によって，その検知が困難になることが指摘されている．本研究では，難読化されたコードを解析・検知に適した形式へ変換するコード正規化モジュールの開発に向けて，難読化がコード構造および実行性能に与える影響を調査する．具体的には，公開されているWasmクリプトジャッキングのバイナリデータを対象に，静的解析によって命令列やコード構造の変化を分析するとともに，難読化前後における計算結果の正当性やハッシュ計算速度などを評価する．その結果，命令数やファイルサイズの変化が，必ずしも実行性能の変化と対応しないことを確認した．これらの結果を踏まえ，コード正規化モジュールの開発に向けた課題を検討する．

キーワード：WebAssembly，クリプトジャッキング，難読化，静的解析，実行性能

Evaluation of Obfuscation Techniques in WebAssembly Cryptojacking Malware

EISUKE KAINUMA[1,a)]　HIROAKI MUKAI[1,b)]

Abstract: WebAssembly (Wasm) enables high-performance computation in web browsers. However, cryptojacking, which exploits this performance to mine cryptocurrency using a device's computing resources without the user's consent, has become a concern. Obfuscation, which makes code structure more complex, has also been reported to hinder detection. Toward developing a code normalization module that transforms obfuscated code into a form suitable for analysis and detection, we investigate the effects of obfuscation on code structure and execution performance. We use publicly available Wasm cryptojacking binaries to analyze changes in instruction sequences and code structure through static analysis and evaluate the correctness of computation results and hashing speed before and after obfuscation. The results confirm that changes in instruction counts and file sizes do not necessarily correspond to changes in execution performance. Based on these findings, we discuss challenges in developing the code normalization module.

1. はじめに

WebAssembly（以下Wasm）は，C/C++などからコンパイルできるバイナリ形式である．ブラウザなどの実行環境で命令と型を検証し，機械語にコンパイルして実行する [1]．表1に，JavaScriptの書き方を制限して高速化したasm.jsとの速度比較を示す．2019年にXeon E5-1650 v3・Ubuntu 16.04で各5回測ったSPEC CPUの結果である [2]．

表 1　asm.jsに対するWasmの速度比（幾何平均）[2]

ブラウザ	Chrome 74.0	Firefox 66.0
速度比	1.54倍	1.39倍

Wasmは，ハッシュ計算を繰り返す暗号資産のマイニングにも使われる．利用者の同意なく端末の計算資源をマイニングに使う攻撃が，クリプトジャッキングである．図1にブラウザ上での攻撃の一例を示す．利用者がWebページを開くと，JavaScriptがWasmを読み込み，マイニングプールとの通信を仲介する．マイニングプールは計算タスクを配分し，送られた結果を確認するサーバである．ブラウザ内のWasmがハッシュ計算を行い，受理された結果に応じて攻撃者が報酬を得る [3]．

難読化は，計算内容を保ちながらコードを読み取りにくくする技術である．知的財産の保護に使われる一方，マルウェアの検知回避にも用いられる [4]．Wasmについても，難読化後のマイニングプログラムが検知を回避する研究結果が報告されている [3], [4]．

難読化でコードの見た目が変わると，既知のマイニングコードとの共通部分を見つけにくくなる．そこで，不要な命令を除き，定数の計算式を簡単にして，比較しやすい形にそろえるコード正規化モジュールを検討する．既知のマイニング処理を特定し，マルウェアの解析・検知を支援することが目的である．

関連する公開データセットには，Webから収集されたWasmバイナリと，研究者が難読化を加えたバイナリが含ま

[1] 金沢工業大学大学院
Graduate School, Kanazawa Institute of Technology
[a)] c6600873@st.kanazawa-it.ac.jp
[b)] mukai.hiroaki@neptune.kanazawa-it.ac.jp

© 2026 Information Processing Society of Japan

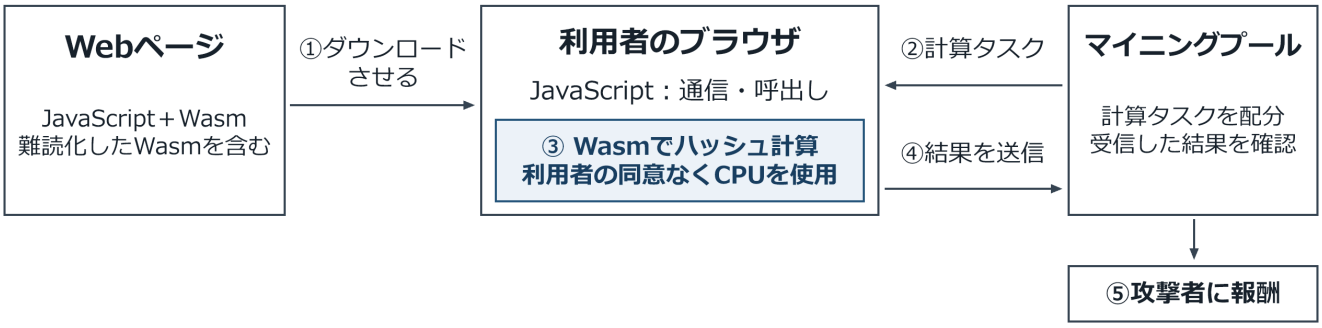


図 1 ブラウザ上のクリプトジャッキングの流れ

れる [4], [5], [6]．対象は，これらに収録された CryptoNight 系列のマイニングコードとする．CryptoNight は暗号資産のマイニングに用いられるハッシュ計算アルゴリズムである．これらを用い，難読化がハッシュ計算の正しさと性能に与える影響，および解析のためにコードを整理できる範囲を評価する．研究課題（RQ）は次の 2 点である．

RQ1 複数のマイニングプログラムを比較し，難読化がハッシュ計算の正しさと計算速度に与える影響の傾向を調べる．また，命令数・ファイルサイズ・機械語コードのサイズと，速度の変化との関係を調べる．

RQ2 コード正規化モジュールと，高速化やコードサイズの削減を目的とする既存の最適化ツールは，難読化後の関数が元のどの関数に当たるかを調べる照合に，どこまで役立つか．複雑な難読化にはどのような課題が残るか．

評価から，命令数の削減量が同じでも，対象関数の実行頻度によって速度変化が異なることを確認した．また，nop 除去と定数畳み込みの併用により，照合対象 644 関数中 536 関数を一意に特定できたが，制御フロー平坦化・命令置換後の XOR 関数 4 件は特定できなかった．これらを基に，性能への影響と既知関数の特定の両面から，コード正規化モジュールの課題を整理する．

2. 関連研究と本研究の位置付け

難読化による速度変化は，既に先行研究で調べられている．Cryptic Bytes データセットを構築した研究は，ソースコード，LLVM の中間表現，Wasm の各段階で難読化を加え，検知への影響，100 秒間のハッシュ計算速度，機械語コードのサイズを評価している [4]．WebAssembly diversification for malware evasion は，難読化前の 6 件と難読化後の 120 件を約 1000 秒間測定し，マイニングプールで計算結果を検証している [3]．MineSweeper も Wasm と asm.js の速度差や CPU 利用率を扱う [5]．

SWAMPED データセットを用いた研究は，JavaScript と Wasm の実行環境である V8 を使い，実行時間を測定している [6]．コンパイラには，コンパイル時間の短い Liftoff を使う．ただし，Wasm が外部に呼び出す関数は，何も処理しない関数に置き換えている．これに対し，今回の実験では JavaScript 側に必要な処理を用意し，計算されたハッシュ値全体を検証する．Proof-of-Theft データセットにも速度測定用プログラムがあるが，本文・補足資料では，ハッシュ値の照合と速度の繰り返し測定を組み合わせた評価は確認できなかった [7]．

先行研究が扱った難読化前後の速度差に対し，RQ1 では関数の実行回数を測り，一部の難読化を手動で除去して速度変化との関係を調べる．RQ2 では，nop 除去と定数畳み込みを既存の最適化と比較し，既知の関数を特定できる範囲を調べる．性能分析と関数照合を組み合わせ，解析・検知を支援する正規化の要件を検討する点が特徴である．新しい難読化方式や正規化規則は提案しない．

3. 対象と分析方法

使用するデータセットは 3 種類である．Cryptic Bytes データセット [4] では CryptoNight 系列の 10 プログラムのコード構造を比較する．実行と関数照合には，Proof-of-Theft データセット [7] の xmr-wasm・webminerpoolhashcn と，SWAMPED データセット [6] のマイニング用プログラム 1 種類を使う．「3 プログラム」はこの 3 種類を指し，難読化などで作る Wasm ファイルの数とは区別する．表 2 に実験別の内訳と目的を示す．

定数分割は固定値を計算式に，命令置換は命令を同じ計算を行う別の命令列に置き換える．次に実行する処理を数値で管理し，処理の順序を追いにくくする難読化を制御フロー平坦化と呼ぶ．

変更前のファイルを「比較元」，難読化を加えたり一部を取り除いたりしたファイルを「変換版」と呼ぶ．比較元も，取得前から難読化されている可能性がある．

xmr-wasm は Monero のマイニング用プログラムである [7]．ハッシュ計算には 8 バイト・16 バイト単位の XOR（排他的論理和）を行う補助関数があり，それぞれ 8 バイト XOR 関数，16 バイト XOR 関数と呼ぶ．

© 2026 Information Processing Society of Japan

表 2　各実験で使う Wasm ファイルの内訳と目的

実験	ファイルの内訳	評価の目的
コード構造の比較	Cryptic Bytes データセット：最適化なし 10,230 ＋あり 10,230 ＝ 20,460（重複を含む）	手法・最適化条件による構造の違い
取得ファイル等の動作・速度確認	xmr-wasm 5 ＋ webminerpoolhashcn 5 ＋ SWAMPED のプログラム 3 ＝ 13	取得したコードの動作・速度の確認
条件をそろえた性能比較	比較元 3 ＋ nop 挿入版 3 ＋定数分割版 3 ＝ 9（3 プログラムに同じ条件で難読化）	ハッシュ値の維持と難読化による速度変化
XOR 関数の復元比較	xmr-wasm の制御フロー平坦化版 1 ＋ XOR 関数の手動復元版 3 ＝ 4	関数の実行頻度と速度変化の関係
コード正規化の評価	条件をそろえた 9 ＋取得した xmr-wasm の制御フロー平坦化版 1・命令置換版 1 ＝ 11	計算結果の維持と既知関数の特定

3.1　コード構造の比較とハッシュ値・速度の確認

Cryptic Bytes データセットの対象 10 プログラムは，CryptoNight v0・v1・v2 やその派生方式の実装である．最適化なし版・あり版は各 10,230 ファイルで，それぞれ比較元 10, LLVM と Tigress による難読化版 160, wasm-mutate による変換系列 10,060 からなる．計 20,460 ファイルには同一内容の重複も含まれる．難読化前後を対応付け，手法と最適化条件によるコード構造の違いを比較する．

ハッシュ値と速度の確認には，表 2 の 13 ファイルを使う．Proof-of-Theft データセットからは，2 プログラムそれぞれについて，比較元と制御フロー平坦化・命令置換・基本ブロック分割・偽の制御フローの 4 設定を取得した（各 5 ファイル，計 10）．SWAMPED データセットから取得した比較元 1 ファイルに nop 挿入版と定数分割版を追加し，残り 3 ファイルとした．取得した 11 ファイルと追加作成した 2 ファイルを含む比較であり，作成条件は統一していない．

Proof-of-Theft データセットには Wasabi で実行状況の記録用命令を加えたファイルもあり，その命令はハッシュ計算や難読化のコードと区別した．

解析ツール WABT 1.0.42 で各ファイルの命令や型が Wasm の規格に合うかを確認し，Wasm のテキスト形式である WAT に変換した．静的解析により，命令数，関数，分岐表，関数呼び出し，ファイル全体と各セクションのサイズを比較した．命令数はコードに書かれた個数であり，同じ命令が繰り返し実行される回数ではない．取得元や設定，比較元との対応も記録した．

ハッシュ計算の正しさは，マイニングソフトウェア XMRig v6.18.0 が公開する CryptoNight v0 のテストデータ 5 件と，その正しいハッシュ値で確認した [8]．xmr-wasm では，対応する 3 件を用いた．マイニングでは，計算に与えるデータ中の調整用の値（ノンス，nonce）を変え，ハッシュ値がしきい値（target）以下になる組合せを探す．xmr-wasm の検証では，しきい値を最大値に設定し，計算中にノンスが変わらないことを確かめて，同じ入力に対するハッシュ値を比較した．webminerpoolhashcn の CryptoNight 派生方式 cn-pico の評価には，マイニングプールに受理された計算結果の保存記録 13 件も用いた．

13 ファイルの速度は，測定前にプログラムを動かすウォームアップを行い，各 3 回測った．webminerpoolhashcn は 1 秒間のウォームアップ後に 5 秒間，他は 3 秒間のウォームアップ後に 30 秒以上測定した．3.3 節のコンパイラを固定した実験とは分けて集計する．実験は，外部通信やマイニングプールへの送信を行わないローカル環境で実施した．

3.2　比較用ファイルの作成と入力条件の確認

条件をそろえるため，3.1 節の 3 プログラムの比較元に，同じ設定で nop 挿入と定数分割を別々に加える．比較元 3 ファイルと新たな難読化版 6 ファイルの計 9 ファイルで，ハッシュ値・速度・関数照合を比べる．3.1 節の 13 ファイルから選ぶのではなく，難読化版を作り直す．

難読化には SWAMPED の変換プログラムを用いた [6]．一つは nop 命令の挿入である．もう一つは，定数を同じ値になる 2 つの整数の加算に置き換える「定数の分割」（Constant Value Splitting）である．例えば 16 を 10 + 6 に置き換える．浮動小数点演算の丸め誤差を避けるため，浮動小数点定数を含む関数は定数分割の対象外とした．

各手法で，関数ごとに変換可能な箇所の約 10%（候補があれば最低 1 箇所）を選んだ．再現できるよう乱数の初期値を固定した．元の Wasm ファイルを WAT に変換し，SWAMPED の処理を適用した後，WABT で Wasm ファイルに戻した．比較元も WAT から Wasm に戻す同じ手順で処理した．したがって，そろえたのは Wasm 上の変換条件であり，元の C/C++を同じ条件でコンパイルし直した比較ではない．

ハッシュ値は，XMRig のテストデータと，ノンスを変えた追加データで確認した．追加データには公開された正解がない．そこで，別の配布元の webminerpoolhashcn を XMRig の正解で検証し，追加データの比較用の値を求めた．比較に使う値を「照合用ハッシュ値」と呼ぶ．ただし，3 プログラムには CryptoNight と暗号処理ライブラリ OpenAES に由来する共通コードがあるため，別の配布元

© 2026 Information Processing Society of Japan

表 3　性能測定の実行環境

項目	設定
CPU	Intel Core i9-11900K
実行環境	Windows 上の Linux コンテナ
Node.js	18.20.4
V8	10.2.154.26-node.37
CPU 割り当て	1 CPU（コンテナ内の CPU 0）
並列実行数	1
メモリ上限	1 GiB
外部通信	無効

との一致だけでは正しさを証明できない．

計算に与えるデータの長さや順序，末尾に残る値を変え，ハッシュ関数がメモリのどこまで読むかも調べた．結果を第 4.2.1 節に示す．以降の実験では，関数が実際に読み込む範囲のデータをそろえる．

3.3　コンパイラ別の速度測定

Wasm の命令数が増えても，コンパイル時に不要な命令が除かれれば，速度への影響は小さくなる可能性がある．この最適化の影響を調べるため，V8 の 2 種類のコンパイラを分けて測定する．Liftoff はコンパイル時間の短さを，TurboFan は実行速度を重視する [9]．それぞれで同じファイルを実行し，難読化前後の速度比を求める．測定中はコンパイラを固定し，Wasm のコンパイルが始まらないことを実行ログで確認した．JavaScript 側の実行時コンパイル（JIT）は確認対象外である．

3 プログラムで CryptoNight v0 のハッシュ計算を繰り返す．3.1 節の webminerpoolhashcn の速度測定は cn-pico という別の計算方式を使うため，両者の速度は直接比較しない．ブラウザを使わず JavaScript を実行できる Node.js に組み込まれた V8 を用いる．表 3 に実行環境を示す．Windows 内の Linux 実行環境（コンテナ）で CPU の割り当てを 1 個に制限したが，他の処理が同じ物理 CPU を使う可能性は残る．

速度測定には，第 3.2 節の 9 ファイルと，XOR 関数の復元比較の 4 ファイルを使う（表 2）．各プログラムの比較元と nop 挿入版・定数分割版を比べ，さらに xmr-wasm の制御フロー平坦化版と，8 バイト XOR のみ・16 バイト XOR のみ・両方の復元版を比べる．計 13 ファイルだが，3.1 節とは内訳が異なる．

1 回の測定では，プログラムを新たに起動し，5 秒以上ウォームアップした後，30 秒以上ハッシュ計算を測る．ハッシュ計算用の領域の初期化・解放と JavaScript からの呼び出し時間を含め，実行環境の準備時間は除く．

測定中の速度変動を確認するため，変換版の前後に比較元を測る．nop 挿入と定数分割の比較では，「比較元，nop 挿入版，比較元，定数分割版，比較元」の 5 回を 1 巡とする．XOR 関数の比較では，3 種類の復元版の間と前後に比較元を測るため，1 巡は 7 回となる．各コンパイラで 6 巡を測り，変換版ごとに 6 回の測定値を得る．6 巡は 2 巡ずつ 3 回に分け，プログラム・変換・コンパイラの測定順を入れ替えた．

表 3　性能測定 272 回の内訳

測定内容	回数の計算	合計
3 プログラムの難読化前後	$5 \times 3 \times 2 \times 6$	180
XOR 関数の復元前後	$7 \times 2 \times 6$	84
比較元の連続測定	–	8
合計		272

表 4 に測定回数を示す．3 プログラムの難読化前後の比較が 180 回，XOR 関数の復元前後の比較が 84 回である．比較元を連続して測り，測定値の揺れを確認する 8 回を加え，合計 272 回となる．

計算速度 H は，1 秒当たりに完了したハッシュ計算数（H/s）であり，計算回数を測定時間で割って求める．変換版の速度 H_V を比べる基準速度 $\widehat{H}_B$ は，前後の比較元の測定値から次式で推定する．

$$\widehat{H}_B = H_1 + \frac{t_V - t_1}{t_2 - t_1}(H_2 - H_1) \tag{1}$$

H_1, H_2 は前後に測った比較元の速度，t_1, t_2, t_V は比較元 2 回と変換版の各測定区間の中央時刻である．式 1 で前後の速度を直線で結び，変換版を測った時点の基準速度を推定する．その基準に対する速度比 r を，

$$r = \frac{H_V}{\widehat{H}_B} \tag{2}$$

で求める．$r = 1$ は同じ速度，$r > 1$ は変換版が高速，$r < 1$ は低速を表す．6 回の速度比は幾何平均でまとめた．一時的な負荷の影響は補間だけでは除けないため，比較元同士の変動も確認し，遅い測定値も除外しなかった．

図 2 の誤差棒は，測定値のばらつきから求めた速度比の推定範囲（95%信頼区間）を参考値として示す．2 巡ずつの速度比を対数にして平均し，得られた 3 つの平均値から t 分布（自由度 2）を使って計算した．各比較で個別に求めた区間であり，全比較をまとめた 95%の信頼水準は保証しない．測定回数が少なく，統計上の前提も十分に確認できないため，区間だけで速度差や同等性は判定しない．

3.4　実行回数の計測と関数別の比較

命令数が増えた関数でも，ハッシュ計算中に使われるとは限らない．速度変化との関係を調べるため，XOR 関数が実際に何回呼ばれるかを測る．対象は，Proof-of-Theft データセットの xmr-wasm の比較元，制御フロー平坦化版，命令置換版の計 3 ファイルである．

制御フロー平坦化された関数は，次の処理を選ぶ値の更新と，共通の選択処理に戻る命令を手掛かりに選定した．分岐表や間接呼び出しは通常の暗号処理にも使われるため，その有無だけでは判断しない．比較元との対応は，関数の

© 2026 Information Processing Society of Japan

表 5　xmr-wasm の計算速度と命令数

難読化の設定	計算速度の中央値（H/s）	最小値〜最大値（H/s）	命令数の比
比較元	9.11	8.50–9.34	1.000
制御フロー平坦化	6.18	5.86–6.22	1.166
基本ブロック分割（設定名）	8.59	8.46–9.01	1.033
命令置換（設定名）	6.72	6.61–6.82	1.655
偽の制御フロー（最適化：-O2）	23.08	22.69–24.05	0.413

番号やラベルに加え，命令列と処理内容から確認した．

各ファイルに，関数の先頭と呼び出し箇所で回数を数えるコードを別々に加えた．XMRig の CryptoNight v0 の対応する 3 件で，二つの数え方が一致することを確認した．

続いて，同じ制御フロー平坦化版から，8 バイト XOR のみ，16 バイト XOR のみ，両方を手動で元に戻した 3 ファイルを作成した．他のコードと，関数の引数・戻り値の型は変えなかった．XMRig のテストデータでハッシュ値の一致を確認し，回数を数えるコードを除いて 3.3 節の方法で復元前後の速度を比べた．

4. 難読化による計算への影響（RQ1）

コード構造とハッシュ計算への影響を分け，条件をそろえた速度測定の結果とともに示す．

4.1 コード構造とハッシュ値・速度の評価

4.1.1 結果

コード構造の比較．Cryptic Bytes データセットの 20,460 ファイルを比較した．最適化なし版では，難読化前を 1 とした命令数比の中央値は，難読化ツール Tigress による算術式の書き換え（encodearithmetic）が 2.515，LLVM を用いた命令の置き換え（subobf）が 1.461，制御フロー平坦化は LLVM が 1.098，Tigress が 1.394 だった．増加量は手法と最適化条件によって異なった．

ハッシュ値・計算速度の評価．3.1 節の 13 ファイルでは，対応する XMRig のテストデータでハッシュ値が照合用ハッシュ値と一致した．表 5 に xmr-wasm の計算速度と命令数を示す．比較元に対する速度比は，制御フロー平坦化版が 0.678，命令置換版が 0.738 だった．SWAMPED データセットを基に 3.1 節で用意した 3 ファイルを各 3 回測ると，nop 挿入版・定数分割版の命令数比は 1.097・1.034，速度の中央値の比は 0.960・0.973 だった．各回では比較元より速い場合と遅い場合があり，比較元自体も 27.09〜32.73 H/s で変動した．

4.1.2 考察

コード構造の比較．命令数の増加量は，難読化手法や最適化条件に左右され，難読化の有無だけでは決まらない．ただし，Cryptic Bytes データセットでの構造比較と 13 ファイルの実行性能の評価は対象が異なるため，両者を同じファイルの結果として結び付けることはできない．

表 6　XOR 関数の実行回数と復元による速度変化

復元対象	1 ハッシュ当たりの呼び出し回数	削減命令数	LF 速度比	TF 速度比
8 バイト XOR	0	91	0.9922	1.0148
16 バイト XOR	1,179,648	91	1.4692	1.3857

ハッシュ値・計算速度の評価．試したテストデータではハッシュ値は保たれた．表 5 では，命令数が多い命令置換版より制御フロー平坦化版が遅く，命令数だけでは速度低下を説明できない．偽の制御フロー版は速かったが，作成時に高速化などを指示する-O2 設定 [10] を使っている．実行時の Liftoff・TurboFan の指定とは異なり，作成条件も未統一のため，難読化と最適化の影響は分けられない．SWAMPED データセットのプログラムでも測定値が変動した．次節では，小さな差を難読化の効果と誤認しないよう，変換条件とコンパイラをそろえる．

4.2 条件をそろえたハッシュ値・速度の評価

4.2.1 結果

ハッシュ値の確認．3.2 節の 9 ファイルで，ハッシュ値が正しいかを確認した．実行方法は，毎回状態を初期化する方法と，状態を保ってデータを順・逆順に与える方法の 3 通りとした．XMRig のテストデータは 2 プログラムで各 5 件，xmr-wasm で 3 件を使い，各 3 種類のファイルを 3 通りの方法で実行した．ハッシュ値の照合は計 117 回（$(5+5+3)\times 3\times 3$）すべてで一致した．さらに，ノンスを変えた 8 件を 9 ファイルで同様に試した 216 回（$8\times 9\times 3$）も一致し，合計 333 回で一致した．

読み込むデータの範囲．ハッシュ関数が実際に読む範囲を調べると，SWAMPED データセットのプログラムは，入力長の引数を別の用途に使い，先頭 76 バイトを読んでいた．長さの指定を変えた 27 条件と，実行履歴や末尾のデータを変えた 81 条件で，76 バイトが同じならハッシュ値は一致した．1 バイトだけ書き込んで長さ 1 を指定しても，残り 75 バイトにある以前のデータまで計算に使われた．webminerpoolhashcn でも，実際に読むデータをそろえた 14 種類でハッシュ値が一致した．さらに，計算に与えるデータを短くした 12 条件でも確認した．

関数の実行回数と復元前後の速度．XMRig の対応するテストデータ 3 件で，xmr-wasm の比較元・制御フロー平坦化版・命令置換版の関数の実行回数を調べた．表 6 にその回数と，制御フロー平坦化版から各 XOR 関数を復元したときの速度比を示す．LF は Liftoff，TF は TurboFan，速度比は復元前に対する 6 回の比の幾何平均である．

16 バイト XOR 関数の復元では，命令数は約 0.05%，機械語コードのサイズは約 0.06〜0.07%減った．6 回の速度比は LF で 1.3819〜1.5327，TF で 1.3227〜1.4690 だった．両方の XOR 関数を復元した場合の速度比は，それぞれ

© 2026 Information Processing Society of Japan

1.4426，1.3450 だった．各復元版は，他の関数と引数・戻り値の型を保ち，XMRig のテストデータ 3 件でハッシュ値が一致した．なお，命令数が増えた箇所には，測定中に使われない別のアルゴリズムの処理も含まれた．

コンパイラ別の速度比較．表 7 に難読化前後の命令数・機械語コードのサイズ・速度の比を，図 2 にコンパイラ別の速度変化を示す．nop 挿入版・定数分割版は作成前の比較元と，XOR 関数の復元版は復元前の制御フロー平坦化版と比べる．速度比は 1 を超えると高速，1 未満なら低速を表す．機械語コードのサイズは V8 の記録値（codesize）の合計であり，Wasm ファイルのサイズとは異なる．速度比は 6 回の幾何平均で示す．図の点は各回の値，ひし形は幾何平均，誤差棒は比較ごとの 95%信頼区間（参考値）である．異なるプログラムの速度そのものは比較しない．nop 挿入で Wasm の命令数は約 9〜10%，ファイルサイズは約 3.7〜4.2%増えた．一方，機械語コードのサイズは Liftoff で約 0.07〜0.31%増え，TurboFan では変化しなかった．SWAMPED データセットのプログラムで定数を分割すると，Liftoff の速度比は 6 回とも 1 未満（0.8511〜0.9554）で，幾何平均は 0.9077 だった．TurboFan では 0.9273〜1.0743 となり，機械語コードのサイズ比も，Liftoff の 1.0486 に対し 1.0009 だった．比較元同士の速度比にも 0.8438〜1.2029 の変動があった．

4.2.2 考察

ハッシュ値と読み出し範囲．試したデータでは難読化後もハッシュ値が一致した．ただし，短いデータを与えた場合も含め，任意の長さのデータで正しく計算できることを示した結果ではない．読み出し範囲が異なれば，ハッシュ値の違いを難読化の影響とは判断できない．長さの引数だけでなく，実際に読まれるデータと初期状態をそろえる必要がある．

実行される関数と速度．表 6 の対比は，同じ 91 命令の削減でも，実行頻度によって速度変化が異なることを示す．頻繁に呼ばれる 16 バイト XOR 関数の復元では高速化したが，呼ばれなかった 8 バイト XOR 関数では同じ改善はなかった．命令数に加え，どの処理が何回実行されるかの確認が必要である．別々に戻した場合の改善分を足しても，両方を戻した場合の改善量にはならなかった．

コンパイラによる違い．表 7 では，Wasm の命令数が増えても，TurboFan の機械語コードのサイズはほとんど変わらなかった．図 2 でも定数分割による速度低下は 3 プログラムに共通せず，実行時の最適化の違いを考慮する必要がある．コードサイズが同じでも機械語が同じとは限らず，追加した nop 命令が除かれたかは未確認である．比較元自体も変動するため，数%の差だけで効果を断定しない．

4.3 RQ1 のまとめ

試したデータではハッシュ値が保たれたが，3 プログラムに共通する速度低下はなかった．命令数・ファイルサイズ・機械語コードのサイズの増減だけでは，速度への影響を判断できない．実行される関数とその回数，コンパイラの条件を踏まえた評価が必要である．

5. コード正規化モジュールの評価（RQ2）

5.1 解析・検知を支援するコード正規化

RQ1 では，コード量だけでは実行性能への影響を判断できなかった．解析・検知の支援には，コードを減らせるかに加え，既知の処理を見つけやすくなるかを確かめる必要がある．そこで，nop 除去と定数畳み込みを行うコード正規化モジュールを試作し，関数照合への効果を評価する．

定数畳み込みは，連続する整数定数の加減乗算・ビット演算に限り，32・64 ビット整数のオーバーフローも，Wasm の規則に従って扱う．メモリ読み出し，関数呼び出し，分岐を含む式は変更しない．比較元のファイルや変換位置の記録は使わず，対象コードだけから書き換え箇所を決める．

正規化の効果は，難読化後の関数から，比較元の関数を特定できるかで評価する．既存の最適化との比較には，Wasm 最適化ツール Binaryen 133 を使う．不要コード除去（vacuum）と定数計算（precompute）の併用，および最適化設定 O2・O3 を試す．O3 は O2 より積極的に最適化する設定である．

5.2 対象ファイルと評価手順

正規化の評価には，第 3.2 節の比較元 3 ファイルと追加した難読化版 6 ファイルに，取得した xmr-wasm の制御フロー平坦化版・命令置換版を加えた計 11 ファイルを使う．XOR 関数の手動復元版は含めない．図 3 のように，両方に同じ正規化を施し，定数の表し方や nop の有無による違いを取り除いて関数を照合する．

11 ファイルのそれぞれに，nop 除去，定数畳み込み，両者の併用，Binaryen の不要コード除去と定数計算の併用，O2，O3 を個別に試す．処理を加えない場合も含めた計 7 条件で，命令・型とハッシュ値を確認する．O3 は先行研究との比較用だが，Binaryen のバージョンが異なるため完全な再現ではない．

照合対象は，追加で難読化した 6 ファイルの 640 関数と，取得した 2 ファイルの XOR 関数 4 件，計 644 関数である．XOR 関数は元との対応を確認済みである．比較元と難読化版に同じ処理を施し，比較元 3 ファイルの全関数から，命令・ローカル変数の宣言・定数などが一致する関数を探す．関数名や関数・呼び出し先の番号は難読化で変わり得るため，照合には使わない．型も番号ではなく内容で比べる．元の関数名は，照合結果が正しいかの確認だけに使う．

正しい関数 1 件だけが候補なら「正解のみ」，正解を含む複数件なら「複数候補」，候補がなければ「一致なし」とする．最適化後の関数が元のどの関数に当たるか確認でき

© 2026 Information Processing Society of Japan

表 7　変更前に対する命令数・コードサイズ・速度の比

対象プログラム	変換	ファイル内の命令数の比	LF コードサイズ比	TF コードサイズ比	LF 速度比	TF 速度比
SWAMPED データセットのプログラム	nop	1.0973	1.0016	1.0000	0.9837	1.0035
	定数の分割	1.0341	1.0486	1.0009	0.9077	1.0121
xmr-wasm	nop	1.0977	1.0007	1.0000	0.9988	0.9859
	定数の分割	1.0136	1.0123	1.0001	1.0014	0.9897
webminerpoolhashcn	nop	1.0914	1.0031	1.0000	0.9908	1.0228
	定数の分割	1.0162	1.0126	1.0018	1.0019	1.0202

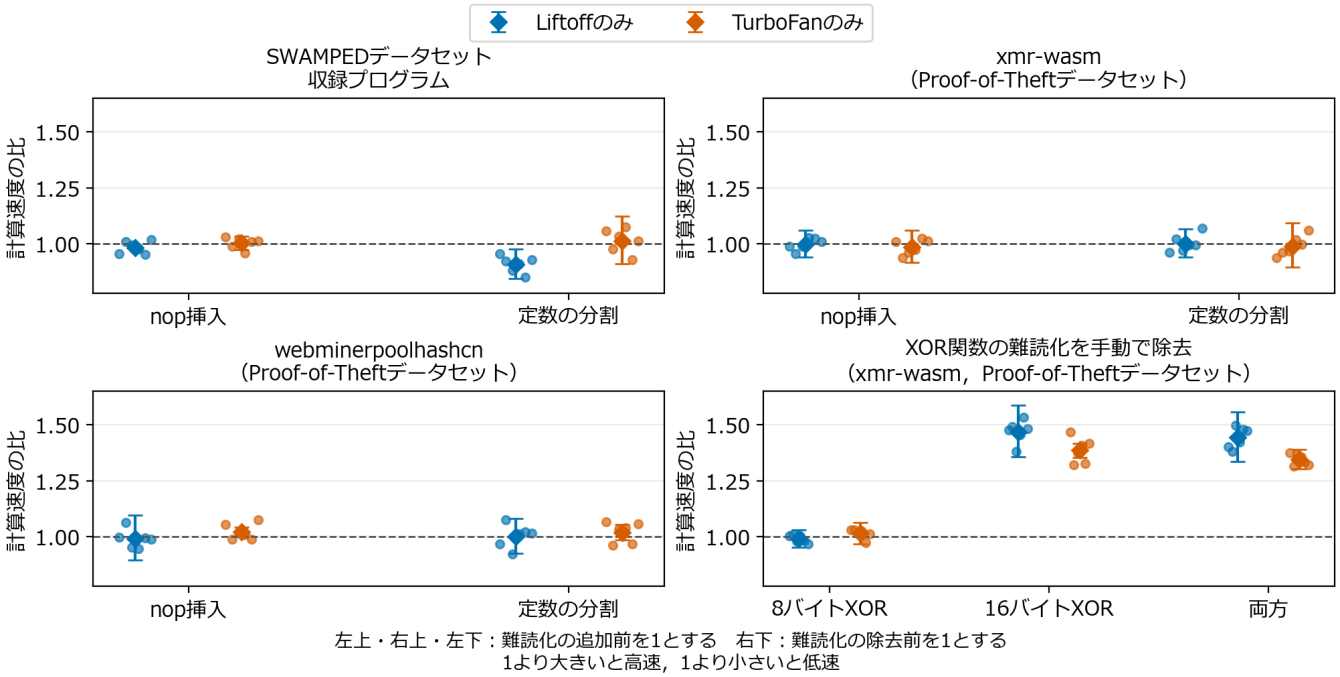


図 2　難読化の追加と一部除去による速度変化

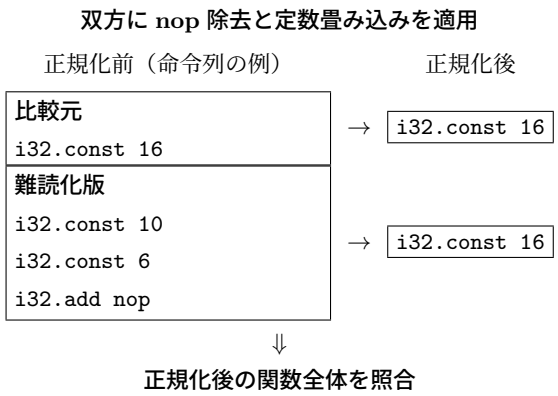


図 3　同じ正規化を施して関数を照合する仕組み

ない場合は「対応不明」とする．

5.3　結果

処理後のファイル． 11 ファイルを，未処理を含む 7 条件で評価したため，結果は計 77 通りとなった．異なる条件でも同じファイルになる場合があり，内容の重複を除くと 42 ファイルだった．全ファイルが WABT による命令・型の検証に合格し，外部から呼ぶ関数の引数・戻り値の型やメモリの条件も保たれた．

ハッシュ値の確認． 状態を毎回初期化する場合と，状態を保ってデータを順・逆順に与える場合を試した．1 ファイルに 1 件のデータを与えてハッシュ値を照合する操作を 1 回と数え，2,793 回すべてで照合用ハッシュ値と一致した．再実行分は含めない．試したデータと実行条件では，計算結果が保たれた．

関数照合． 表 8 に，644 関数を 7 条件で照合した結果（計 4,508 回）を示す．nop 除去と定数畳み込みを併用すると，追加で難読化した 640 関数のうち 536 関数で，対応する比較元の関数を 1 件に絞れた．取得した難読化済みの XOR 関数 4 件は，どの処理でも一致しなかった．全 644 関数に対しては 83.2%を 1 件に絞れた．これは既知の nop 挿入・定数分割に対する関数照合の結果であり，未知の難読化にも対応できることを示すものではない．また，マルウェアの検知率とは異なる．誤った関数だけが候補に残る例はなかった．

O2・O3 では，それぞれ 287・300 関数で元との対応を追えなかった．関数そのものが削除・統合されたのか，対応の確認に失敗したのかは，完全には区別できなかった．正

© 2026 Information Processing Society of Japan

表 8　難読化の種類別に見た関数照合結果

処理	正解のみ	複数候補	一致なし	対応不明
追加した nop 挿入・定数分割（640 関数）				
未処理	27	22	591	0
nop 除去	295	71	274	0
定数畳み込み	268	55	317	0
nop 除去と定数畳み込み	536	104	0	0
Binaryen 不要コード除去＋定数計算	294	68	278	0
Binaryen O2	274	18	61	287
Binaryen O3	249	22	69	300
取得した制御フロー平坦化・命令置換（XOR 関数 4 件）				
全 7 通り	0	0	4	0

規化後の速度は未測定である．

5.4　コード正規化モジュールの効果と課題

表 8 では，nop 除去と定数畳み込みの併用が，単独処理や試した Binaryen の設定より多くの関数を一意に特定した．ただし 104 関数では候補が複数残った．呼び出し先の番号を照合に使わないことが一因と考えられ，関数間の呼び出し関係で候補を絞ることが課題である．

既存の最適化では，関数の削除・統合やインライン展開により，元の関数を追いにくくなる場合があった．命令列を比べる今回の照合では，計算内容が同じでも，関数の分け方や命令の並びが変わると一致しない場合がある．

制御フロー平坦化・命令置換の自動解除は未実装で，これらを施した XOR 関数は特定できなかった．未評価のプログラムでも照合できるか，解析時間や検知性能を改善できるかの検証が必要である．

6.　限界と再現性

結果を一般化できる範囲． 3 プログラムのハッシュ値・性能・関数照合の結果は，構造比較に使った Cryptic Bytes データセット全体には当てはめられない．対象は CryptoNight 系列，PC・Node.js は各 1 種類で，元の C/C++ から Wasm を作る条件も未統一である．同一内容のファイルや共通コードを含むため，ファイル数は独立したプログラムの数を表さない．

評価できていないこと． ハッシュ値の一致は試したデータと初期状態での確認にとどまり，計算結果以外にメモリなどへ与える影響も含め，動作全体が同じとは証明していない．Wasm のコンパイル記録から，JavaScript の実行時コンパイルやコード配置の影響は分からない．また，関数の実行回数は処理時間の割合を示さない．XOR 関数の手動復元と自動正規化の効果は区別する．実際の攻撃での難読化の普及率も未評価である．解析・検知への課題は 5.4 節に述べた．

再現性のための記録． ファイルの SHA-256 値，変換条件，測定順序，コンパイル記録，実行ログ，集計コードを保存した．予備測定 72 回のうち，TurboFan も使われた 24 回は，Liftoff のみで測った結果の集計から除外した．

7.　まとめ

解析・検知の支援に向け，難読化の性能への影響（RQ1）と，正規化による既知関数の特定（RQ2）を評価した．RQ1 では，試したデータでハッシュ値は保たれたが，同じ命令数の削減でも，対象関数の実行頻度によって速度変化が異なった．RQ2 では，nop 除去と定数畳み込みの併用で 644 関数中 536 関数を一意に特定できた一方，複雑な難読化を施した XOR 関数 4 件は特定できなかった．

この二つの結果から，コード正規化は，計算結果を保つことを確認した上で，コード量や速度の変化と，既知の処理を特定できるかを分けて評価する必要がある．手動復元による高速化は，自動正規化の効果を示すものではない．今後は複雑な難読化の解除と関数照合を改善し，未評価のプログラムで解析時間や検知性能への効果を確かめる．

参考文献

[1] WebAssembly: WebAssembly, https://webassembly.org/. Accessed 2026-10-06.
[2] Jangda, A., Powers, B., Berger, E. D. and Guha, A.: Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code, *Proc. USENIX ATC 2019*, USENIX Association, pp. 107–120 (2019).
[3] Cabrera-Arteaga, J., Monperrus, M., Toady, T. and Baudry, B.: WebAssembly diversification for malware evasion, *Computers & Security*, Vol. 131, p. 103296 (2023).
[4] Harnes, H. and Morrison, D.: Cryptic Bytes: WebAssembly Obfuscation for Evading Cryptojacking Detection, arXiv:2403.15197v1 (2024).
[5] Konoth, R. K., Vineti, E., Moonsamy, V., Lindorfer, M., Kruegel, C., Bos, H. and Vigna, G.: MineSweeper: An In-depth Look into Drive-by Cryptocurrency Mining and Its Defense, *Proc. ACM CCS 2018*, ACM, pp. 1714–1730 (2018).
[6] Kim, T., Oh, S., Lee, K., Wang, W., Kwon, Y., Hong, S. and Kim, H.: When Does Wasm Malware Detection Fail? A Systematic Analysis of Their Robustness to Evasion, *Proc. IEEE/ACM ASE 2025*, IEEE, pp. 2957–2969 (2025).
[7] Sermchaiwong, T. and Shen, J.: Proof-of-Theft: Dynamic Graph-Based Fingerprinting of In-Browser Cryptomining, *Proc. ECOOP 2026*, LIPIcs, Vol. 372, pp. 25:1–25:31 (2026).
[8] XMRig developers: CryptoNight test vectors, XMRig v6.18.0, https://github.com/xmrig/xmrig/blob/v6.18.0/src/crypto/cn/CryptoNight_test.h. Accessed 2026-10-08.
[9] V8 team: Liftoff: a new baseline compiler for WebAssembly in V8, https://v8.dev/blog/liftoff (2018). Accessed 2026-10-25.
[10] Emscripten contributors: Optimizing Code, https://emscripten.org/docs/optimizing/Optimizing-Code.html. Accessed 2026-10-25.

© 2026 Information Processing Society of Japan